

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (5,6)

Αναζήτηση και ταξινόμηση

Γραφήματα

23/4/2024

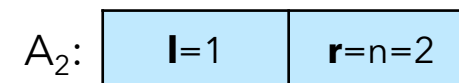
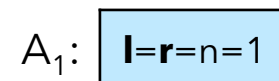
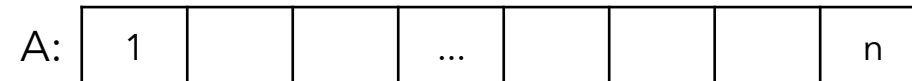
Έφη Μαλέσιου
Διδακτορική φοιτήτρια

Αλγόριθμος 3 Stooge Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```
1: if  $A[l] > A[r]$  then  
2:   Swap( $A[l], A[r]$ )  
3: end if  
4: if  $l + 1 \geq r$  then  
5:   return  
6: end if  
7:  $k := \lfloor (r - l + 1)/3 \rfloor$   
8: Stooge Sort ( $A, l, r - k$ )  
9: Stooge Sort ( $A, l + k, r$ )  
10: Stooge Sort ( $A, l, r - k$ )
```



α Αποδείξτε ότι η κλήση Stooge Sort($A, 1, n$) ταξινομεί σωστά ένα πίνακα A μήκους n .

β Δώστε μία αναδρομική εξίσωση που να περιγράφει το χειρότερο χρόνο εκτέλεσης του αλγορίθμου και την ακριβή πολυπλοκότητα του χρόνου αυτού (σε συμβολισμό Θ).

Αλγόριθμος 3 Stooage Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```

1: if  $A[l] > A[r]$  then

```

```
2:  Swap( $A[l], A[r]$ )
```

```
3: end if
```

4: **if** $l + 1 \geq r$ **then**

```
5: return
```

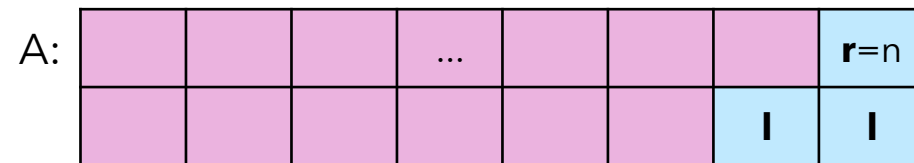
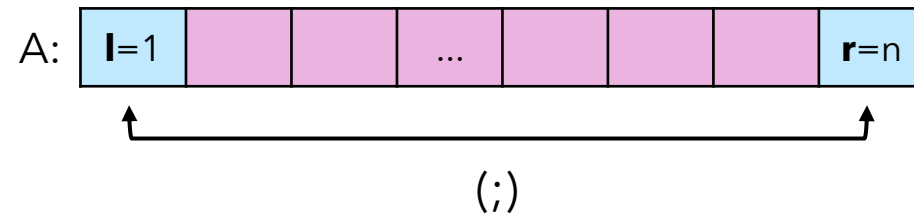
```
6: end if
```

$$7: k := \lfloor (r - l + 1)/3 \rfloor$$
8: Stooge Sort ($A, l, r - k$)

```

9: Stooge Sort ( $A, l + k, r$ )

```

10: Stooge Sort ($A, l, r - k$)

A:	l=1		k=3						r=9
A ₁ :	l=1					r-k=6	k θέσεις		
A ₂ :	κ θέσεις			l+k=4					r=9
A ₃ :	l=1					r-k=6	k θέσεις		

Αλγόριθμος 3 Stooge Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```
1: if  $A[l] > A[r]$  then  
2:   Swap( $A[l], A[r]$ )  
3: end if  
4: if  $l + 1 \geq r$  then  
5:   return  
6: end if  
7:  $k := \lfloor (r - l + 1)/3 \rfloor$   
8: Stooge Sort ( $A, l, r - k$ )  
9: Stooge Sort ( $A, l + k, r$ )  
10: Stooge Sort ( $A, l, r - k$ )
```

A:	l=1		k=3						r=9
A ₁ :	l=1					r-k=6	k θέσεις		
A ₂ :	k θέσεις			l+k=4					r=9
A ₃ :	l=1					r-k=6	k θέσεις		

$k \approx n/3$

- $l=1, r=n$
- l έως $r-k, \sim n-n/3 = 2n/3$ στοιχεία

Αλγόριθμος 3 Stooge Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```
1: if  $A[l] > A[r]$  then  
2:   Swap( $A[l], A[r]$ )  
3: end if  
4: if  $l + 1 \geq r$  then  
5:   return  
6: end if  
7:  $k := \lfloor (r - l + 1) / 3 \rfloor$   
8: Stooge Sort ( $A, l, r - k$ )  
9: Stooge Sort ( $A, l + k, r$ )  
10: Stooge Sort ( $A, l, r + k$ )
```

όσα από τα k
μεγαλύτερα του
 A στον A_1

StoogeSort($A, l, r-k$)

A:	l=1		k=3						r=9
A ₁ :	l=1			k+1		r-k=6	k θέσεις		
A ₂ :	k θέσεις			l+k=4					r=9
A ₃ :	l=1					r-k=6	k θέσεις		

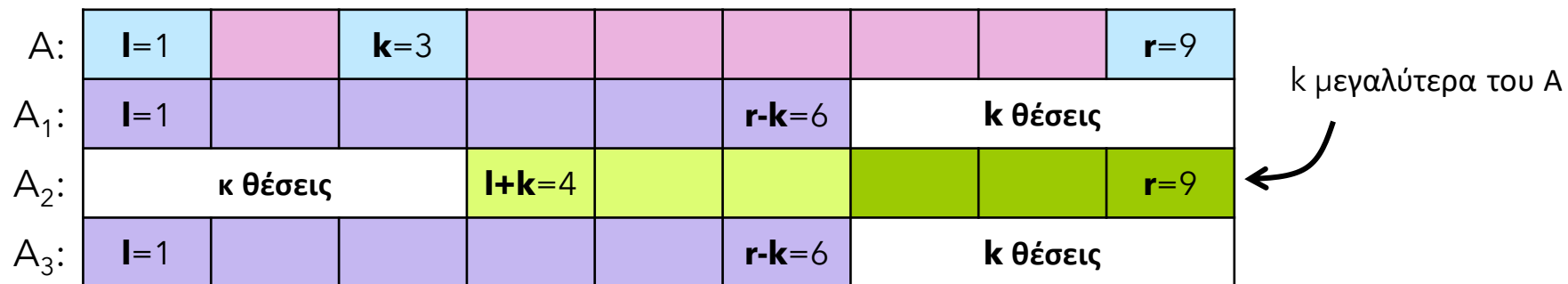
Αλγόριθμος 3 Stooge Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```
1: if  $A[l] > A[r]$  then  
2:   Swap( $A[l], A[r]$ )  
3: end if  
4: if  $l + 1 \geq r$  then  
5:   return  
6: end if  
7:  $k := \lfloor (r - l + 1) / 3 \rfloor$   
8: Stooge Sort ( $A, l, r - k$ )  
9: Stooge Sort ( $A, l + k, r$ )  
10: Stooge Sort ( $A, l, r - k$ )
```

StoogeSort($A, l+k, r$)



Αλγόριθμος 3 Stooge Sort (A, l, r)

Είσοδος: Πίνακας n στοιχείων A και δείκτες l, r

Έξοδος: Ταξινομημένη έκδοση του A από το δείκτη l έως το δείκτη r

```
1: if  $A[l] > A[r]$  then  
2:   Swap( $A[l], A[r]$ )  
3: end if  
4: if  $l + 1 \geq r$  then  
5:   return  
6: end if  
7:  $k := \lfloor (r - l + 1) / 3 \rfloor$   
8: Stooge Sort ( $A, l, r - k$ )  
9: Stooge Sort ( $A, l + k, r$ )  
10: Stooge Sort ( $A, l, r - k$ )
```

A:	l=1		k=3						r=9
A ₁ :	l=1					r-k=6	k θέσεις		
A ₂ :	k θέσεις			l+k=4					r=9
A ₃ :	l=1					r-k=6	k θέσεις		

StoogeSort($A, l, r-k$)

ταξινόμηση μικρότερων
στοιχείων



Λύση. α. Απόδειξη με επαγωγή:

Βάση επαγωγής: Ο αλγόριθμος δουλεύει σωστά για μήκος πίνακα 1 ή 2.

Επαγωγική Υπόθεση: Έστω ότι ο αλγόριθμος δουλεύει σωστά για κάθε μήκος πίνακα μικρότερο από n .

Βήμα επαγωγής: Θα δείξουμε ότι ο αλγόριθμος δουλεύει σωστά για μήκος πίνακα $n + 1$.

Χωρίς βλάβη της γενικότητας, θα δείξουμε ότι η κλήση $\text{Stooge Sort}(A, 1, n + 1)$ δουλεύει για έναν πίνακα A με $n' = n + 1$ στοιχεία. (Στη γενική περίπτωση πρέπει να δείξουμε ότι η κλήση $\text{Stooge Sort}(A, l, n - l)$ δουλεύει, αλλά η απόδειξη γίνεται με ακριβώς τον ίδιο τρόπο). Επίσης χωρίς βλάβη της γενικότητας, θα πάρουμε $n \geq 2$ (έχουμε ήδη δείξει ότι δουλεύει για $n = 1$). Τότε $k = \lfloor (n + 1)/3 \rfloor \geq \lfloor 3/3 \rfloor = 1$.

Οι τρεις μεταγενέστερες κλήσεις της Stooge Sort καλούνται σε μήκος πίνακα $n + 1 - k \leq n$, επομένως από την επαγωγική υπόθεση ταξινομούν σωστά τους υποπίνακές τους.

Η πρώτη κλήση της Stooge Sort , ταξινομεί τον υποπίνακα από 1 έως $n' - k$. Ας ονομάσουμε αυτόν τον υποπίνακα A_1 . Τα $n' - 2k$ μεγαλύτερα στοιχεία αυτού του πίνακα, μετά την ταξινόμηση βρίσκονται στις θέσεις $k + 1..n' - k$. Παρατηρούμε ότι $n'/3 \geq k \Rightarrow n' - 2k \geq k$, οπότε, ξέρουμε ότι τα k μεγαλύτερα στοιχεία του A_1 βρίσκονται στις θέσεις $k + 1..n' - k$.

Επίσης, ο A_1 είναι υποπίνακας του A . Άρα, όσα από τα k μεγαλύτερα στοιχεία του A βρίσκονται στον A_1 , αναγκαστικά ανήκουν στα k μεγαλύτερα στοιχεία του A και βρίσκονται στις θέσεις $k + 1..n' - k$ μετά την πρώτη ταξινόμηση.

Η δεύτερη κλήση της Stooge Sort, ταξινομεί τον υποπίνακα από $k + 1$ έως n' . Ας τον πούμε A_2 . Ο A_2 περιέχει και τα k μεγαλύτερα στοιχεία του A , αφού όλα όσα ήταν στον A_1 βρίσκονται μετά την πρώτη κλήση στις θέσεις $k + 1..n' - k$, ενώ τα υπόλοιπα είναι στο κομμάτι $n' - k..n'$ του A_2 . Άρα, μετά τη δεύτερη ταξινόμηση, τα k μεγαλύτερα στοιχεία του A βρίσκονται στις θέσεις $n' - k + 1..n'$ και είναι ταξινομημένα. Επομένως, τα $n' - k$ μικρότερα στοιχεία του A βρίσκονται στον υποπίνακα A_1 . Επομένως, για να ταξινομηθεί ολόκληρος ο A , αρκεί να ταξινομηθεί ο A_1 , το οποίο και κάνει η τρίτη κλήση της Stooge Sort.

β. Ο αλγόριθμος χωρίζει τον πίνακα σε τρία μέρη, μήκους $k, n - 2k, k$ αντίστοιχα, όπου $k = \lfloor n/3 \rfloor$. Το μεσαίο τμήμα είναι κατά 0,1 ή 2 στοιχεία μεγαλύτερο από τα άλλα δύο, κάτι που ασυμπτωτικά δεν έχει σημασία. Μπορούμε λοιπόν να πούμε ότι ο αλγόριθμος χωρίζει τον πίνακα σε τρία μέρη, μήκους $n/3$. Αφού επιτελέσει εργασία σταθερής πολυπλοκότητας, έστω 1, καλεί τρεις φορές τον εαυτό του, σε πίνακες μήκους $2n/3$. Επομένως, η αναδρομή που δίνει την πολυπλοκότητα είναι

$$T(n) = 3T(2n/3) + 1$$

Εφαρμόζοντας τη μέθοδο Master σε αυτήν την αναδρομή, με $a = 3$ και $b = 3/2$, παίρνουμε:

$$n^{\log_b a} = n^{\log_{1.5} 3}$$

Είναι

$$1 = O(n^{\log_{1.5} 3 - \epsilon})$$

για $\epsilon \leq \log_{1.5} 3$. Επομένως $T(n) = \Theta(n^{\log_{1.5} 3})$. Η πολυπλοκότητα αυτή ισχύει για κάθε περίπτωση (η χειρίστη, η βέλτιστη και η μέση συμπίπτουν).

Δίνονται n βίδες & n αντίστοιχα παξιμάδια, διαφορετικού διαμετρήματος.

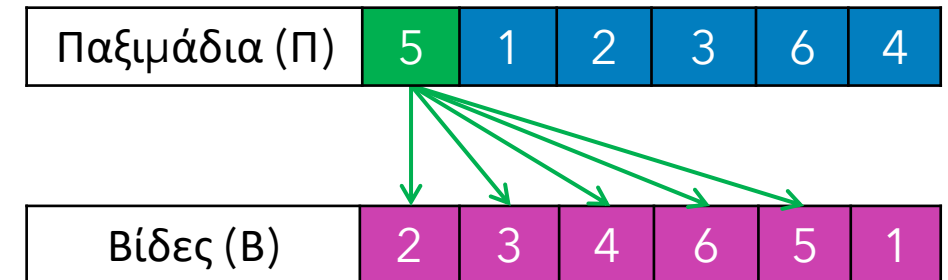
Μπορεί να ελεγχθεί αν ένα επιλεγμένο ζεύγος βίδας και παξιμαδιού είναι αντίστοιχα, με 1 δοκιμή ταιριάσματος.

- Σχεδιάστε αλγόριθμο κατάλληλου ταιριάσματος των βιδών με τα παξιμάδια, με αποδοτικότητα μέσης περίπτωσης $\Theta(n \log n)$.



Προσέγγιση 1:

- Αλγόριθμος **εξαντλητικής αναζήτησης**
 - Εδώ, συγκρίνουμε κάθε βίδα με όλα τα παξιμάδια μέχρι να βρούμε η συγκεκριμένη βίδα σε ποιο παξιμάδι ταιριάζει.
 - Έτσι, μία βίδα θα κάνει έως n συγκρίσεις με τα παξιμάδια. Επομένως θα γίνουν έως n συγκρίσεις $(\#\text{βιδών}) * n (\#\text{παξιμαδιών}) = n^2$ συγκρίσεις.
 - Αρα, ο χρόνος εκτέλεσης του αλγορίθμου είναι **$O(n^2)$** .



Προσέγγιση 2:

- Αλγόριθμος **διαίρει και βασίλευε**
 - Στηρίζεται στην προσέγγιση της τυχαιοποιημένης γρήγορης ταξινόμησης (τυχαία επιλογή *pivot* – Randomized QuickSort)
 - Διαλέγουμε **τυχαία** ένα **παξιμάδι**, από τον πίνακα των παξιμαδιών Π , $\Pi[i]$ και αυτό χρησιμοποιείται ως **pivot** για την αναδιάταξη του **πίνακα B** των **βιδών** κάνοντας από 1 δοκιμή ταιριάσματος και σχηματίζοντας τελικά 3 ομάδες βιδών: αυτές που χρειάζονται μικρότερο παξιμάδι, αυτές που χρειάζονται παξιμάδι ίδιο με το *pivot* και αυτές που χρειάζονται μεγαλύτερο παξιμάδι. Γνωρίζουμε ότι τουλάχιστον μία βίδα θα ταιριάζει με το εκάστοτε παξιμάδι.
 - Η **βίδα που ταιρίαξε** στο παξιμάδι-*pivot*, γίνεται **pivot** για τον πίνακα των **παξιμαδιών** και ακολουθείται η ίδια διαδικασία όπως και στο προηγούμενο βήμα.
 - Επαναλαμβάνονται τα 2 παραπάνω βήματα **σε κάθε υπο-πίνακα** που δημιουργείται έως την **τελική ταξινόμησή** τους, οπότε η **βίδα** που βρίσκεται σε κάθε θέση του τελικού ταξινομημένου πίνακα βιδών, έχει το **αντίστοιχο παξιμάδι** στην **ίδια θέση** του τελικού ταξινομημένου πίνακα των παξιμαδιών.

Προσέγγιση 2:

- Αλγόριθμος **διαίρει και βασίλευε**
 - Η διάσπαση γίνεται σε χρόνο $\Theta(n)$, όπου διαχωρίζεται το αρχικό πρόβλημα σε 2 υπο-προβλήματα με τυχαίο κάθε φορά μέγεθος και ακολουθείται η διεργασία του Randomized Quicksort, που γνωρίζουμε ότι έχει με μεγάλη πιθανότητα, αναμενόμενο χρόνο εκτέλεσης $\Theta(n \log n)$

Υποθέστε ότι διαθέτετε κάποιο σύστημα επεξεργασίας πληροφοριών το οποίο χρησιμοποιεί την Quicksort για να επεξεργαστεί δεδομένα που λαμβάνει συνδεδεμένο σε ένα δίκτυο. Επιθυμείτε να προστατεύσετε το σύστημά σας από την πιθανότητα να πέσει σε 'σαμποτάζ' από εχθρικές συνδέσεις - θα μπορούσε να λάβει δεδομένα ειδικώς διαμορφωμένα ώστε η μέθοδος Quicksort να εκδηλώσει την χειρότερη επίδοσή της και έτσι το σύστημά σας να σπαταλήσει άσκοπα πόρους ή/και επιπλέον υπολογιστική ισχύ (κάπως έτσι ενδεχομένως να αναγκαζόταν να εγκαταλείψει άλλες ωφέλιμες συνδέσεις).

1. Υποθέτοντας ότι η Quicksort απλώς διαλέγει το πρώτο στοιχείο ως pivot, τι δεδομένα θα μπορούσε να στείλει κάποιος κακόβουλος ώστε να κακομεταχειριστεί το σύστημά σας;
2. Προτείνετε μια απλή στρατηγική (γραμμικού χρόνου) για να αποφύγετε το πρόβλημα που επιχειρεί να δημιουργήσει στο σύστημά σας κάποιος επιτηθέμενος. Δηλαδή μια στρατηγική που εγγυάται ότι η Quicksort θα εκτελεστεί σε χρόνο $O(n \log n)$, ασχέτως αν τα δεδομένα είναι κατασκευασμένα με κακόβουλο σκοπό. Δεν επιτρέπεται να αλλάξετε τον τρόπο που η Quicksort επιλέγει το pivot στοιχείο.

- Η χειρότερη περίπτωση, προκύπτει με ταξινομημένη είσοδο, οπότε με αυτό το pivot δε θα οδηγηθούμε στη δημιουργία δύο υπο-πινάκων μεγέθους $\sim n/2$ (n το μέγεθος του αρχικού πίνακα).
- Μπορούμε να «ανακατέψουμε» την είσοδο, π.χ. με τον αλγόριθμο Fisher-Yates:
 - for i from 0 to $(n-2)$
 - $j = \text{random int from } i \text{ to } n$
 - $\text{swap}(\text{array}[i], \text{array}[j])$

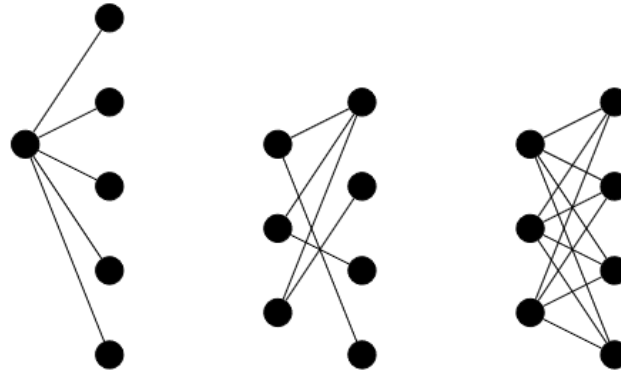
διμερής γράφος

$G = (V, E)$

$V = V1 + V2, V1 \cap V2 = \emptyset$

$V1, V2$: μερικά σύνολα

ακμές: από/προς το $V1$, προς/από το $V2$



k -κανονικός γράφος

ίδιος βαθμός (k) για κάθε κορυφή

κλίκα

ακμή μεταξύ κάθε ζεύγους κορυφών

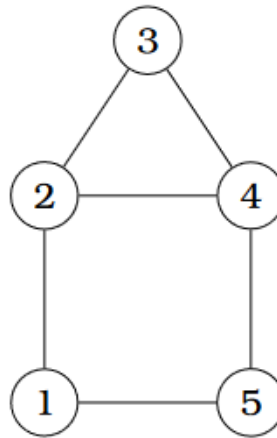
μεγιστική κλίκα

δεν μπορεί να προστεθεί άλλη κορυφή στην αρχικ

κλίκα και να εξακολουθεί να είναι κλίκα

μέγιστη κλίκα

κλίκα με τον max. #κορυφών



κλίκες:

$\{1,2\}, \{1,5\}, \{2,3\}, \{2,4\}, \{3,4\}, \{4,5\}, \{2,3,4\}$

μεγιστικές κλίκες:

$\{1,2\}, \{1,5\}, \{4,5\}, \{2,3,4\}$

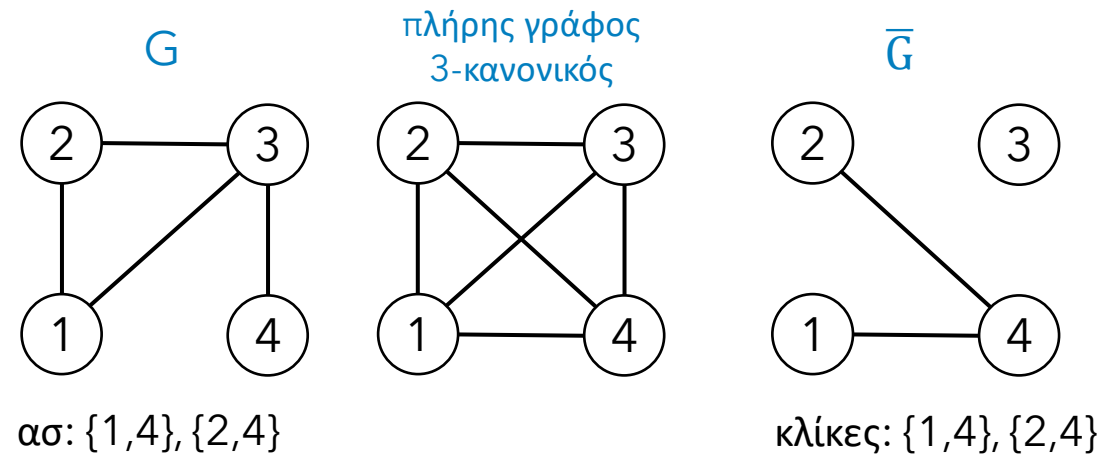
μέγιστη κλίκα:

$\{2,3,4\}$

ανεξάρτητο σύνολο (ασ)

σύνολο κορυφών που ανά 2 δεν έχουν ακμή μεταξύ τους

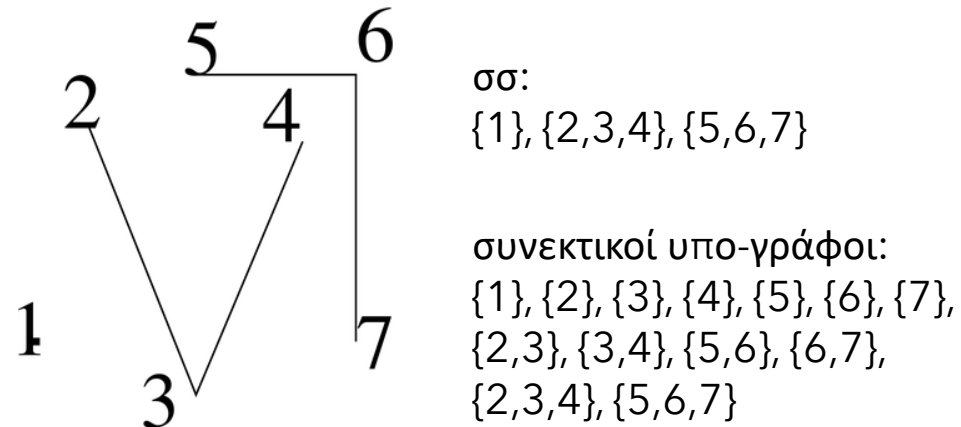
το ανεξάρτητο σύνολο στον G αποτελεί κλίκα στον συμπληρωματικό γράφο ($\bar{G}$ = πλήρης γράφος - ακμές G)



συνεκτική συνιστώσα (σσ)

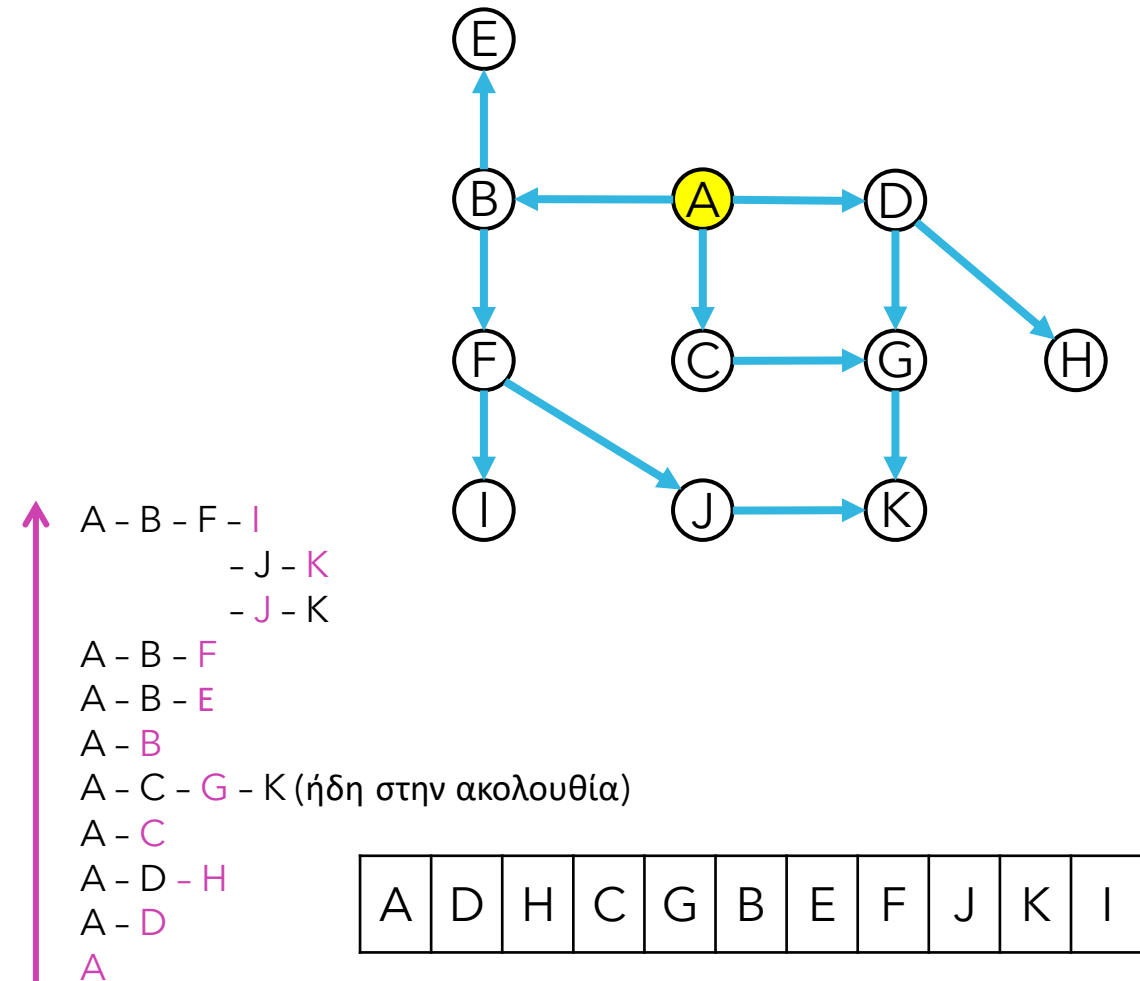
μεγιστικός συνεκτικός υπο-γράφος

μονοπάτι μεταξύ κάθε ζεύγους των κορυφών του



Τοπολογική ταξινόμηση

- μοντελοποίηση σχέσεων εξάρτησης με κατευθυνόμενο ακυκλικό γράφο (DAG - Directed Acyclic Graph)
- με ποια σειρά πρέπει να εκτελεστούν
- DFS
- ακολουθία κορυφών όπου προηγούνται αυτές από τις οποίες εξαρτώνται μετέπειτα διαδικασίες
 - όχι μοναδική
- μερική διάταξη => καθολική διάταξη
- $\Theta(V + E)$
- π.χ. χρήση για την εύρεση μονοπατιού **μεγαλύτερου κόστους** ξεκινώντας από δεδομένη κορυφή **s**

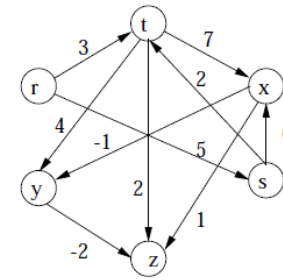


Τοπολογική ταξινόμηση

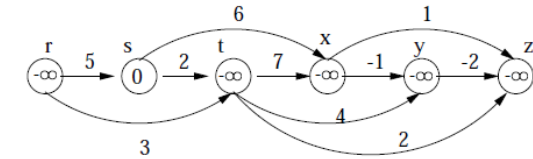
Εύρεση μονοπατιού μεγαλύτερου κόστους

(κατευθυνόμενος, ακυκλικός γράφος με βάρη)

- αρχικά όλες οι αποστάσεις προς άλλες κορυφές: $-\infty$
- τοπολογική ταξινόμηση (ττ) κορυφών
- για κάθε κορυφή **u** της ττ:
 - για κάθε γειτονική κορυφή **v** της **u**
 - αν $\text{dist}[\mathbf{v}] < \text{dist}[\mathbf{u}] + \text{weight}(\mathbf{u}, \mathbf{v})$
 - $\text{dist}[\mathbf{v}] = \text{dist}[\mathbf{u}] + \text{weight}(\mathbf{u}, \mathbf{v})$



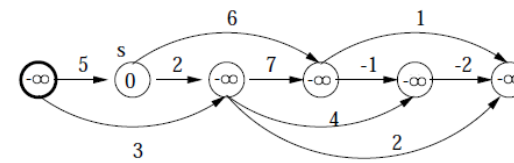
(a)



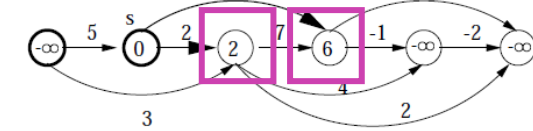
τοπολογική ταξινόμηση

$r - s - t - x - y - z$

$$\begin{aligned} \text{dist}[t] &= \text{dist}[s] + \text{weight}(s, t) = 0 + 2 = 2 \\ \text{dist}[x] &= \text{dist}[s] + \text{weight}(s, x) = 0 + 6 = 6 \end{aligned}$$

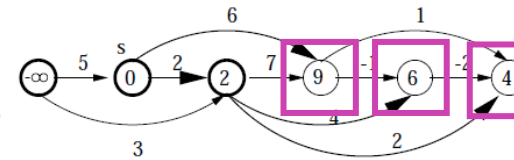


(c)

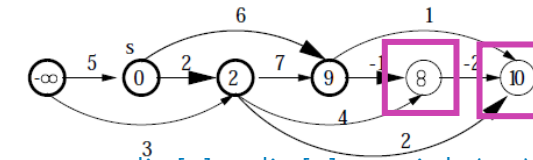


(d)

$$\begin{aligned} \text{dist}[x] &= \text{dist}[t] + \text{weight}(t, x) = 2 + 7 = 9 \\ \text{dist}[y] &= \text{dist}[t] + \text{weight}(t, y) = 2 + 4 = 6 \\ \text{dist}[z] &= \text{dist}[t] + \text{weight}(t, z) = 2 + 2 = 4 \end{aligned}$$

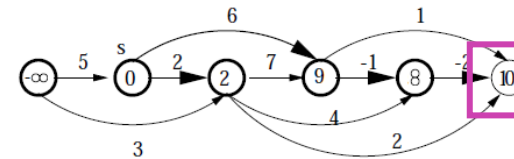


(e)

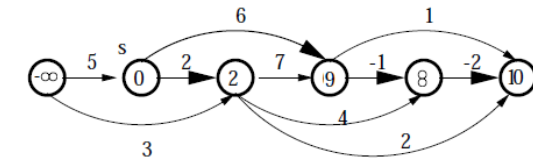


$$\begin{aligned} \text{dist}[y] &= \text{dist}[x] + \text{weight}(x, y) = 9 - 1 = 8 \\ \text{dist}[z] &= \text{dist}[x] + \text{weight}(x, z) = 9 + 1 = 10 \end{aligned}$$

$$\text{dist}[y] + \text{weight}(y, z) = 8 - 2 = 6 < 10$$



(g)

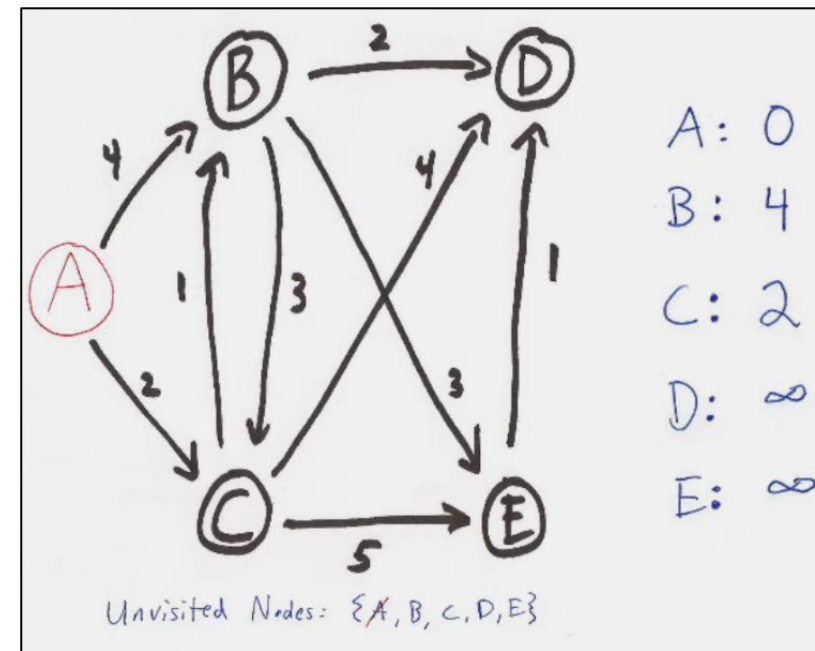
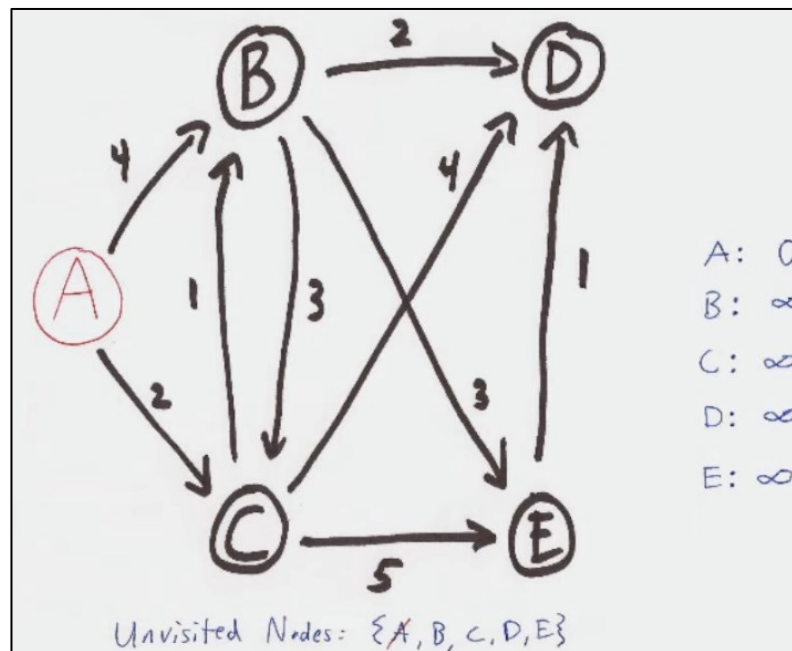
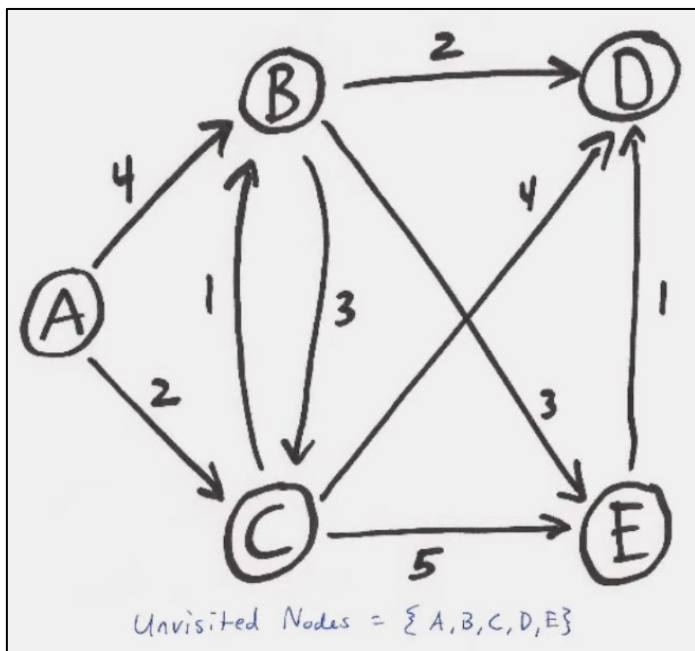


(h)

Αλγόριθμος Dijkstra

σε κατευθυνόμενο γράφο με μη-αρνητικά βάρη

Συντομότερο μονοπάτι από 1 κόμβο προς κάθε άλλο

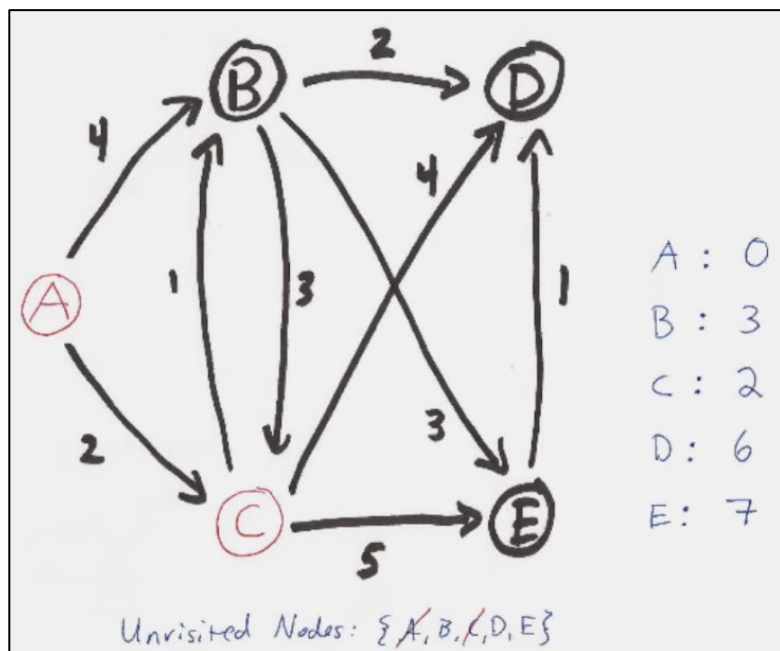


πού φτάνουμε με το 1^ο βήμα;

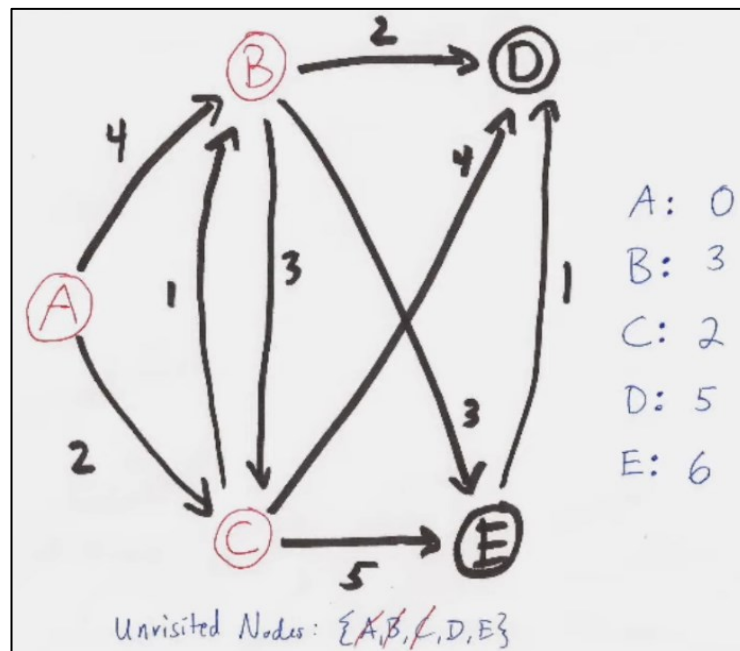
Αλγόριθμος Dijkstra

σε κατευθυνόμενο γράφο με μη-αρνητικά βάρη

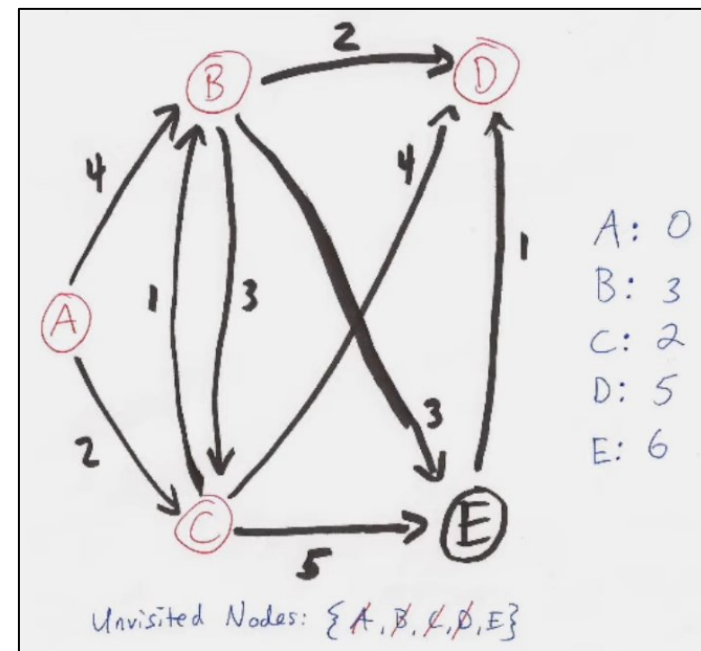
Συντομότερο μονοπάτι από 1 κόμβο προς κάθε άλλο



πού φτάνουμε με min. κόστος; C
νέες κορυφές που προστίθενται (ναι),
ανανέωση βαρών (ναι)



πού φτάνουμε με min. κόστος; B
νέες κορυφές που προστίθενται (όχι),
ανανέωση βαρών (ναι)

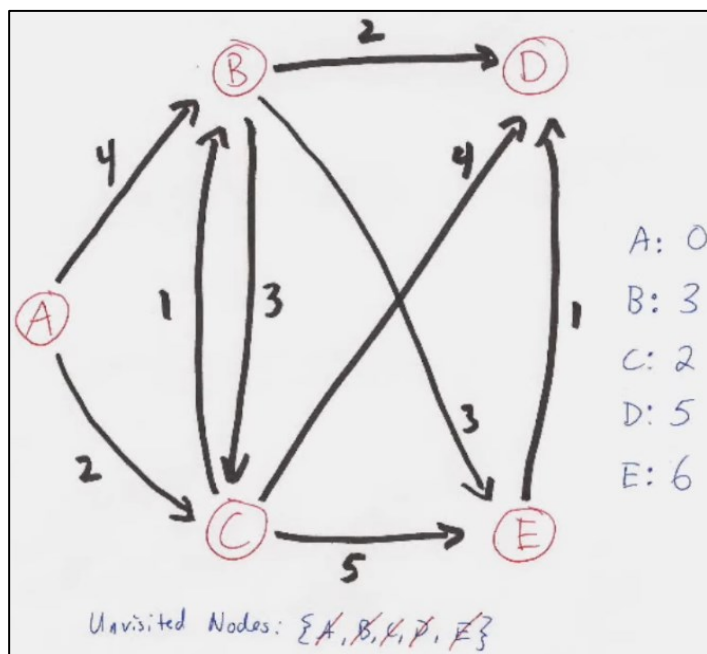


πού φτάνουμε με min. κόστος; D
νέες κορυφές που προστίθενται (όχι),
ανανέωση βαρών (όχι)

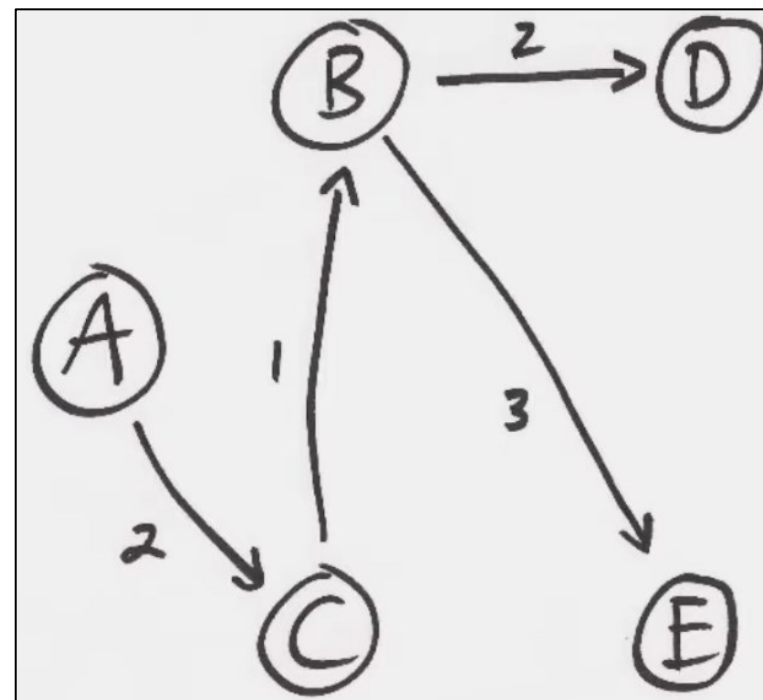
Αλγόριθμος Dijkstra

σε κατευθυνόμενο γράφο με μη-αρνητικά βάρη

Συντομότερο μονοπάτι από 1 κόμβο προς κάθε άλλο



πού φτάνουμε με min. κόστος; E
νέες κορυφές που προστίθενται (όχι),
ανανέωση βαρών (όχι)

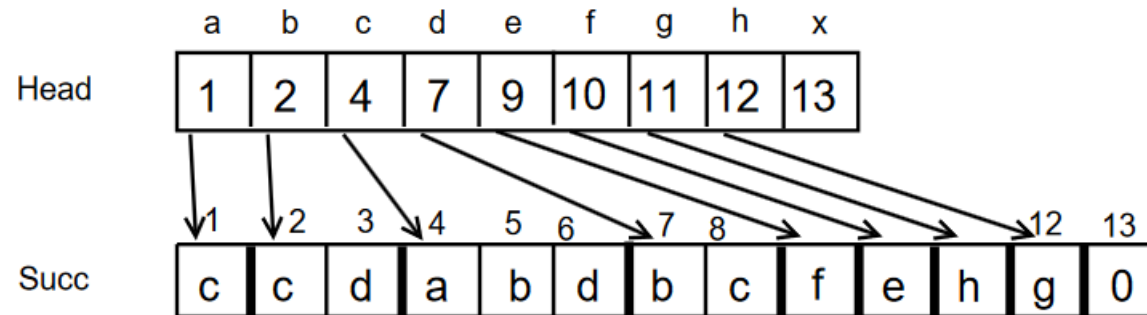


συντομότερα μονοπάτια από το A
 $O(|E| + |V| \cdot \log |V|)$

4. Υπολογισμός συνεκτικών συνιστωσών

Να βρεθούν οι συνεκτικές συνιστώσες του γράφου που παριστάνεται με λίστα γειτνίασης μέσω των γραμμικών πινάκων *Head* και *Succ*:

Ποια είναι η πολυπλοκότητα του αλγορίθμου εύρεσης των συνεκτικών συνιστωσών;



Λύση.

(Ο γράφος που παριστάνουν οι λίστες Head και Succ είναι αυτός του σχήματος 10.)

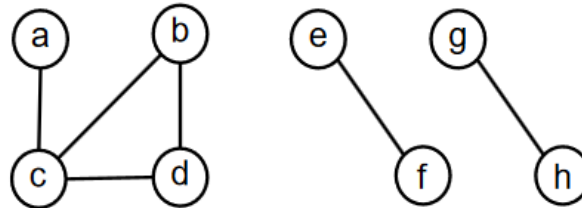
Τις συνεκτικές συνιστώσες ενός γράφου $G = (V, E)$ τις βρίσκουμε με τον αλγόριθμο 1 (BFT - Breadth First Graph Transversal).

Στην περίπτωση μας καλείται αρχικά μια *BFS* με αρχή τον κόμβο a . Αυτή η *BFS* θα ανακαλύψει τους κόμβους b, c και d , άρα όλοι αυτοί μαζί αποτελούν μια συνεκτική συνιστώσα. Στη συνέχεια καλείται μια *BFS* με αρχή τον κόμβο e που ανακαλύπτει μόνο τον κόμβο f , και αυτή είναι η δεύτερη συνεκτική συνιστώσα. Τέλος καλείται μια *BFS* με αρχή τον κόμβο g που ανακαλύπτει τον κόμβο h και αυτή είναι η τρίτη συνεκτική συνιστώσα του γράφου. Δεν υπάρχουν άλλοι αμαρκάριστοι κόμβοι στο γράφο, οπότε ο αλγόριθμος έχει βρει όλες τις συνιστώσες και τερματίζει.

Η πολυπλοκότητα του αλγορίθμου είναι

$$\Theta(n) + \sum_{i=1}^k \Theta(n_i + m_i) = \Theta(n) + \Theta(n + m) \approx \Theta(n + m),$$

όπου k το πλήθος των συνεκτικών συνιστωσών του γράφου και n_i, m_i το πλήθος των κόμβων και των ακμών της i -οστής συνεκτικής συνιστώσας αντίστοιχα.



Σχήμα 10: Ο γράφος που παριστάνουν οι λίστες Head και Succ.

Αλγόριθμος 1 BFT (G)

```
1: Θεωρούμε ένα πίνακα mark με  $|V|$  στοιχεία  
2: Αρχικοποιούμε τον mark σε false  
3: for  $i = 1$  to  $n$  do  
4:   if  $mark[v_i] == false$  then  
5:      $BFS(v_i)$   
6:   end if  
7: end for
```

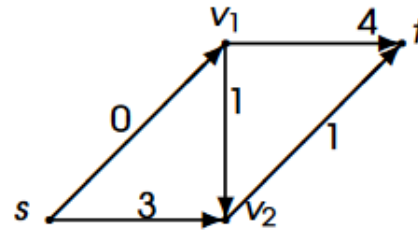

Σε μια περιοχή συγκρούσεων ή φυσικών καταστροφών, μια αποστολή πρέπει να δρομολογηθεί από μια πόλη s σε μια πόλη t . Το οδικό δίκτυο της περιοχής είναι περιγεγραμμένο από ένα γράφο $G = (X, A, P)$, όπου X το σύνολο των πόλεων και A το σύνολο των δρόμων μεταξύ των πόλεων. Για κάθε δρόμο (i, j) , η τιμή $P(i, j)$ είναι η πιθανότητα να διασχισθεί χωρίς επιπτώσεις ο δρόμος από την πόλη i στην πόλη j . Ζητάμε τον καθορισμό ενός δρομολόγιου που μεγιστοποιεί την πιθανότητα να φτάσει η αποστολή από την πόλη s στην t , δηλαδή ένα μονοπάτι μέγιστης αξιοπιστίας.

1. Επιλέξτε τον κατάλληλο αλγόριθμο για την επίλυση του προβλήματος.

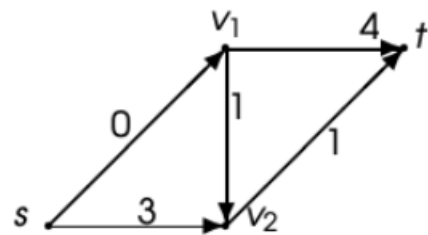
2. Εφαρμόστε τον αλγόριθμο σας στο γράφο με:

- $X = \{s, v_1, v_2, t\}$
- $A = \{(s, v_1), (s, v_2), (v_1, v_2), (v_1, t), (v_2, t)\}$
- $P(s, v_1) = 1, P(s, v_2) = \frac{1}{8}, P(v_1, v_2) = P(v_2, t) = \frac{1}{2}, P(v_1, t) = \frac{1}{16}$

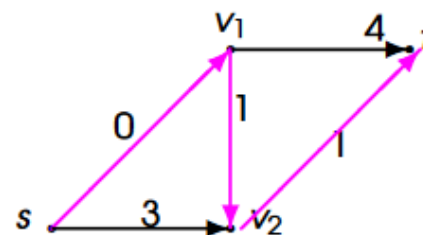
Θα χρησιμοποιήσουμε τον αλγόριθμο του *Dijkstra*, αφού αλλάξουμε το βάρος κάθε πλευράς του γράφου από p σε $-\log p$.
Εφαρμόζοντας την ιδέα μας καταλήγουμε στο γράφο:



Dijkstra					
		s	v_1	v_2	t
Βήμα 1	Fix	1	0	0	0
	P	s	s	s	0
	V	0	0	3	∞
Βήμα 2	Fix	1	1	0	0
	P	s	s	v_1	v_1
	V	0	0	1	4
Βήμα 3	Fix	1	1	1	0
	P	s	s	v_1	v_2
	V	0	0	1	2
Βήμα 4	Fix	1	1	1	1
	P	s	s	v_1	v_2
	V	0	0	1	2

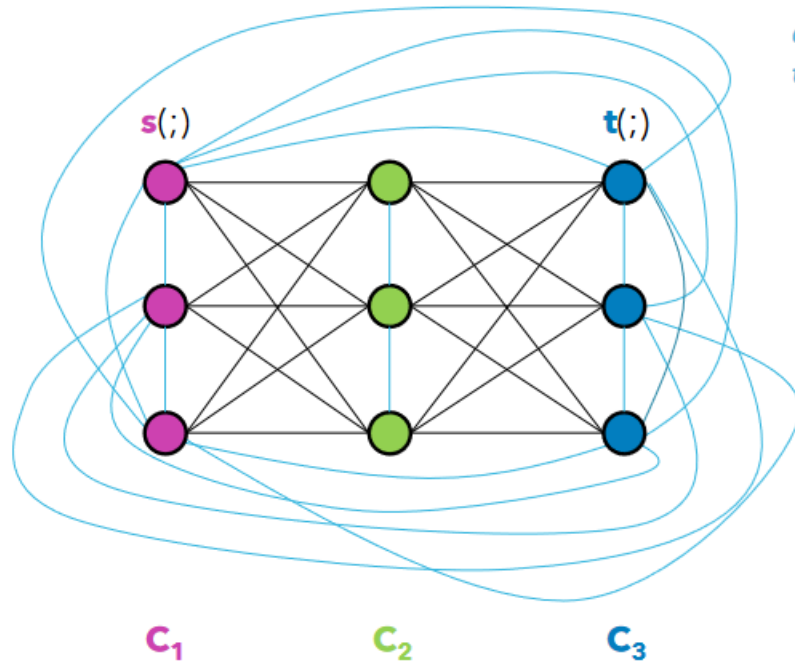


Το συντομότερο μονοπάτι μας έχει βάρος 2 και είναι το s, v_1, v_2, t .
 Άρα το μονοπάτι μέγιστης αξιοπιστίας είναι το s, v_1, v_2, t με αξιοπιστία $1/4$.

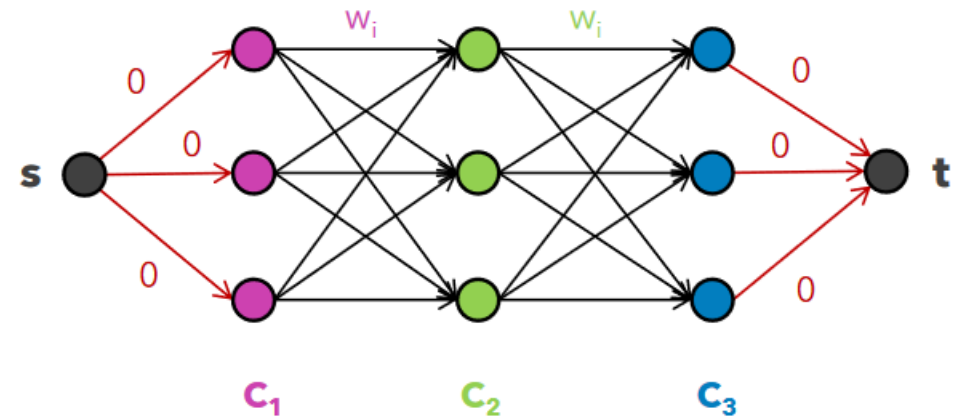


Έστω $G = (V, E, W)$ συνεκτικός, μη κατευθυνόμενος και πλήρης γράφος, όπου $W : E \rightarrow \mathbb{R}$. Θεωρούμε $C_1, C_2, \dots, C_k \subseteq V$, $C_i \neq \emptyset$, $\forall i$, τα οποία δεν περιέχουν κοινά στοιχεία μεταξύ τους, δηλαδή $C_i \cap C_j = \emptyset$, $\forall i \neq j$. Να σχεδιάσετε πολυωνυμικό αλγόριθμο (δίνοντας και την αντίστοιχη πολυπλοκότητα του αλγορίθμου) ο οποίος να βρίσκει ένα μονοπάτι ελαχίστου μήκους k κορυφών τής μορφής $c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_k$, $c_i \in C_i \forall i$ (δηλαδή το πρώτο στοιχείο να ανήκει στο σύνολο C_1 , το δεύτερο στο C_2 κ.λπ.).

Παράδειγμα:
 $n = 9, k = 3$



οι μπλε ακμές δε
θα εξεταστούν



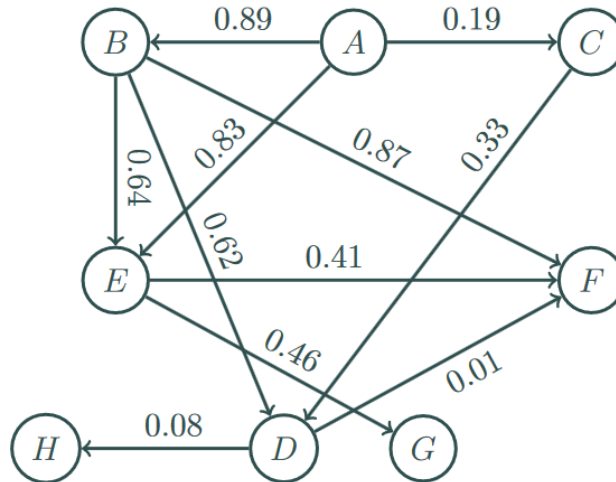
3 κορυφές * 3 βάρη ακμών + 3 κορυφές * 3 βάρη ακμών

Εφόσον ο γράφος είναι πλήρης, γνωρίζουμε ότι υπάρχει ακμή μεταξύ κάθε ζεύγους κορυφών. Θα λάβουμε υπόψη μας μόνο τις ακμές που συνδέουν κορυφές με το ένα άκρο τους στο C_i και το άλλο στο C_{i+1} , $1 \leq i \leq (k-1)$ (έστω $E_{i,i+1}$, το εκάστοτε υποσύνολο ακμών), εφόσον θέλουμε στο μονοπάτι που θα δημιουργηθεί από k κορυφές καθεμία να ανήκει στα διατεταγμένα υποσύνολα C_i . Επιπλέον, πρέπει το k να έχει τιμή ≥ 2 καθώς αν $k = 1$ δεν προκύπτει μονοπάτι. Ο γράφος μπορεί να μετασχηματιστεί σε κατευθυνόμενο, διατηρώντας μόνο τις προαναφερθείσες ακμές που λαμβάνουμε υπόψη μας και με κατεύθυνση από κορυφή του υποσυνόλου C_i προς κορυφή του υποσυνόλου C_{i+1} . Επίσης, προστίθεται μία κορυφή έναρξης s , η οποία συνδέεται με κάθε κορυφή του C_1 με κατευθυνόμενη ακμή βάρους 0 (από την s προς αυτές του C_1), και επιπλέον προστίθεται μία κορυφή - προορισμός, t , η οποία συνδέεται με κάθε κορυφή του C_k με κατευθυνόμενη ακμή βάρους 0 (από τις κορυφές του C_k προς την t).

Με αυτόν τον τρόπο έχει δημιουργηθεί ένας ακυκλικός κατευθυνόμενος γράφος, με συγκεκριμένη κορυφή έναρξης και συγκεκριμένη κορυφή πέρατος, με μόνα δυνατά μονοπάτια μεταξύ τους, αυτά που διέρχονται από μία κορυφή και μόνο κάθε υποσυνόλου διατηρώντας τη διάταξή τους. Αυτός ο μετασχηματισμός, καθώς και οι μηδενικού βάρους ακμές που προστίθενται, δεν επηρεάζουν το κόστος των εφικτών λύσεων του αρχικού γράφου.

Για την εύρεση του μονοπατιού ελάχιστου κόστους, μεταξύ των έγκυρων βάσει των ζητούμενων, αρκεί η χρήση της τοπολογικής ταξινόμησης (άμεση καθώς είναι διατεταγμένα τα υποσύνολα κορυφών C_i και επιπλέον ξεκινάμε με πλήρη γράφο, οπότε είναι: $s, C_1, \dots, C_k, t$) και η ανανέωση του κόστους μετάβασης σε κάθε κορυφή και για την τελική απάντηση απλώς αφαιρούμε τις κορυφές s, t . Για κάθε κορυφή της τοπολογικής ταξινόμησης (KTT), εξετάζουμε μεταξύ των γειτονικών της, αν το προηγούμενο κόστος έως αυτές είναι μεγαλύτερο από αυτό για την KTT + το βάρος της ακμής που τις συνδέει με την KTT, οπότε και ανανεώνεται το κόστος με τη νέα ελάχιστη τιμή. Αυτό έχει ως αποτέλεσμα, όταν ολοκληρωθεί η διαδικασία, το συνολικό κόστος για την t να είναι το ελάχιστο δυνατό. Η διαδρομή δίνεται όπως και στο Θ2, κρατώντας σε έναν πίνακα, τις κορυφές που μείωσαν τελευταία φορά το κόστος για εκάστοτε θέση. Η πολυπλοκότητα της διαδικασίας, είναι: $\max\{O(|V| + |E|), O(|V|)\} = O(|V| + |E|) = O(|V| + |V|^2) = O(|V|^2)$, που είναι πολυωνυμική. Παρατηρούμε ότι εξετάζουμε $\sum_{i=1}^{k-1} (|C_i| \cdot |C_{i+1}|) + O(n)$ ακμές και για $k = 3$, με $|C_1| = |C_2| = |C_3| = \frac{n}{3}$, έχουμε $2\frac{n^2}{9} + 2\frac{n}{3} = \Theta(n^2)$ ακμές με $(n + 2)$ κορυφές, οπότε διαπιστώνουμε ότι δεν προκύπτει με αυτόν τον τρόπο πιο 'στενό' άνω όριο.

Δίνεται ένας κατευθυνόμενος γράφος $G = (V, E)$, με βάρη για κάθε ακμή $(u, v) \in E, r(u, v)$, τα οποία είναι πραγματικές τιμές και επιπλέον ισχύει, $0 \leq r(u, v) \leq 1$. Το βάρος της ακμής, συμβολίζει την αξιοπιστία του αντίστοιχου διαύλου επικοινωνίας από την κορυφή u στην κορυφή v , ως η πιθανότητα να μην αποτύχει η μετάδοση του μηνύματος μεταξύ τους, υποθέτοντας επιπλέον ότι οι πιθανότητες αυτές είναι ανεξάρτητες. Προσδιορίστε δύο αποδοτικούς αλγορίθμους για την εύρεση του πιο αναξιόπιστου μονοπατιού από δεδομένη κορυφή s , θεωρώντας πως δεν υπάρχουν κύκλοι· στον έναν θα αντιμετωπιστεί αμιγώς ως πρόβλημα εύρεσης του μονοπατιού μέγιστου κόστους και στον άλλον ως πρόβλημα εύρεσης του μονοπατιού ελάχιστου κόστους, με κατάλληλους μετασχηματισμούς. Ποια είναι η πολυπλοκότητα χρόνου καθενός; Εφαρμόστε τον αλγόριθμό σας για την εύρεση του μονοπατιού μέγιστου κόστους στον ακόλουθο γράφο, δείχνοντας τα βήματα που ακολουθεί, με έναρξη από την κορυφή A :



Με τη θεώρηση του πιο αναξιόπιστου μονοπατιού ως αυτού όπου δε μεταδίδεται το μήνυμα από ένα τουλάχιστον τμήμα του:

Η ζητούμενη πιθανότητα, είναι συμπληρωματική αυτής της επιτυχίας σε όλα τα τμήματα.

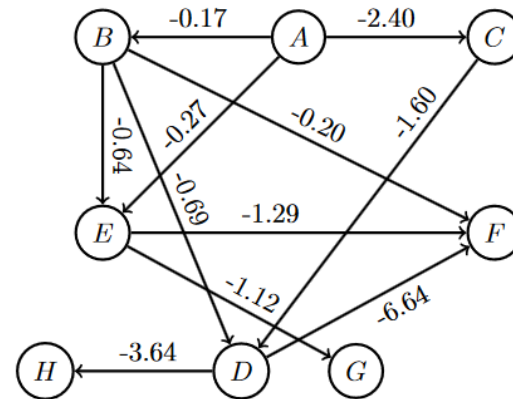
1. Πρόβλημα εύρεσης μονοπατιού μικρότερου κόστους:

Επειδή $\log(\prod_i r_i) = \sum_i \log(r_i)$, μετασχηματίζουμε τα βάρη των ακμών αποδίδοντάς τους την τιμή $\log(r(u, v))$.

Με αυτόν τον τρόπο, θέλουμε να βρούμε πότε το γινόμενο των r_i ελαχιστοποιείται, οπότε η συμπληρωματική πιθανότητα για αυτό το μονοπάτι, που θα είναι η πιθανότητα αναξιοπιστίας, γίνεται μέγιστη οδηγώντας στην επιλογή του ζητούμενου μονοπατιού, και έτσι το πρόβλημα ανάγεται στην εύρεση του συντομότερου μονοπατιού από την κορυφή s προς οποιαδήποτε άλλη.

Για αυτόν τον σκοπό μπορεί αρχικά να γίνει τοπολογική ταξινόμηση, με πολυπλοκότητα $\Theta(|V| + |E|) = O(|V| + |V|^2) = O(|V|^2)$, και στη συνέχεια επισκεπτόμαστε τις κορυφές βάσει της τοπολογικής ταξινόμησης, έχοντας αρχικοποιήσει τις ελάχιστες αποστάσεις προς κάθε κόμβο με τιμή $+\infty$. Για κάθε γειτονική κορυφή v της τρέχουσας κορυφής u , ελέγχεται αν η τρέχουσα τιμή του ελάχιστου κόστους έως την v είναι μεγαλύτερη από το άθροισμα του ελάχιστου κόστους έως την κορυφή u με την προσθήκη του βάρους της ακμής (u, v) , περίπτωση κατά την οποία της αποδίδουμε αυτήν την τιμή. Αυτή η διαδικασία έχει πολυπλοκότητα χρόνου $O(|V|^2)$, αφού για κάθε κορυφή ελέγχουμε $O(|V|)$ ακμές. Επιπλέον, διατηρούμε σε έναν πίνακα με μία θέση για κάθε κορυφή, το όνομα της κορυφής που τελευταία φορά άλλαξε την τιμή της ελάχιστης απόστασής της, προκειμένου να αναδομηθεί στη συνέχεια το πλήρες μονοπάτι ελάχιστου κόστους, με πολυπλοκότητα $O(|V|)$, αφού θα διατρέξουμε το πολύ $|V|$ κορυφές στην προς τα πίσω πορεία και ο εντοπισμός της μικρότερης τιμής μεταξύ των τελικών απαιτεί το πολύ τη διάτρεξη μία φορά του πίνακα τελικών τιμών (χωρίς να εξετάζουμε την τιμή 0 για την κορυφή έναρξης, A), με πολυπλοκότητα $O(|V|)$. Έτσι, η πολυπλοκότητα χρόνου συνολικά, είναι $\max\{O(|V|^2), O(|V|^2), O(|V|)\} = O(|V|^2)$, αφού εκτελούνται διαδοχικά, και είναι αποδοτικός αφού έχει ρυθμό αύξησης μικρότερο από ή ίσο με μία πολυωνυμική με μικρό σχετικά βαθμό.

Σχήμα 4: Μετασχηματισμένα βάρη ($\log(r(u, v))$)



2. Πρόβλημα εύρεσης μονοπατιού **μεγαλύτερου** κόστους:

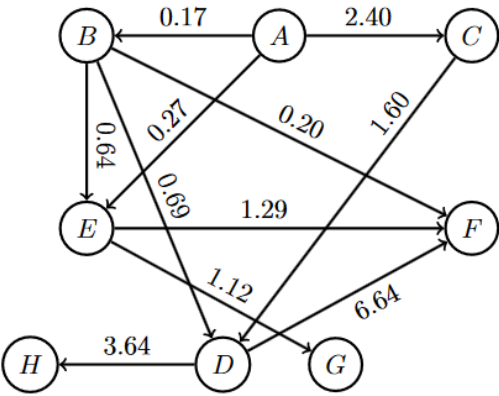
Επειδή $\log(\prod_i r_i) = \sum_i \log(r_i)$, μετασχηματίζουμε τα βάρη των ακμών αποδίδοντάς τους την τιμή $-\log(r(u, v))$.

Με αυτόν τον τρόπο, θέλουμε να βρούμε πότε το γινόμενο των r_i ελαχιστοποιείται, το οποίο ισοδυναμεί με τη μεγιστοποίηση του λογαρίθμου του με αρνητικό πρόσημο (μιας και η $\log$ είναι αύξουσα συνάρτηση) ή του αθροίσματος των λογαρίθμων με αρνητικό πρόσημο των επιμέρους αριθμών, από την κορυφή s προς οποιαδήποτε άλλη.

Για αυτόν τον σκοπό μπορεί αρχικά να γίνει τοπολογική ταξινόμηση, με πολυπλοκότητα $\Theta(|V|+|E|) = O(|V|+|V|^2) = O(|V|^2)$, και στη συνέχεια επισκεπτόμαστε τις κορυφές βάσει της τοπολογικής ταξινόμησης, έχοντας αρχικοποιήσει τις μέγιστες αποστάσεις προς κάθε κόμβο με τιμή $-\infty$. Για κάθε γειτονική κορυφή v της τρέχουσας κορυφής u , ελέγχεται αν η τρέχουσα τιμή του μέγιστου κόστους έως την v είναι μικρότερη από το άθροισμα του μέγιστου κόστους έως την κορυφή u με την προσθήκη του βάρους της ακμής (u, v) , περίπτωση κατά την οποία της αποδίδουμε αυτήν την τιμή. Αυτή η διαδικασία έχει πομπυλοκότητα χρόνου $O(|V|^2)$, αφού για κάθε κορυφή ελέγχουμε $O(|V|)$ ακμές. Επιπλέον, διατηρούμε σε έναν πίνακα με μία θέση για κάθε κορυφή, το όνομα της κορυφής που τελευταία φορά άλλαξε την τιμή της μέγιστης απόστασής της, προκειμένου να αναδομηθεί στη συνέχεια το πλήρες μονοπάτι μέγιστου κόστους, με πολυπλοκότητα $O(|V|)$, αφού θα διατρέξουμε το πολύ $|V|$ κορυφές στην προς τα πίσω πορεία και ο εντοπισμός της μέγιστης τιμής μεταξύ των τελικών απαιτεί το πολύ τη διάτρεξη μία φορά του πίνακα τελικών τιμών (χωρίς να εξετάζουμε την τιμή 0 για την κορυφή έναρξης, A), με πολυπλοκότητα $O(|V|)$. Έτσι, η πολυπλοκότητα χρόνου συνολικά, είναι $\max\{O(|V|^2), O(|V|^2), O(|V|)\} = O(|V|^2)$, αφού εκτελούνται διαδοχικά, και είναι αποδοτικός αφού έχει ρυθμό αύξησης μικρότερο από ή ίσο με μία πολυωνυμική με μικρό σχετικά βαθμό.

Βήματα:

Σχήμα 5: Μετασχηματισμένα βάρη $(-\log(r(u, v)))$



Τοπολογική ταξινόμηση, με αναζήτηση πρώτα κατά βάθος (DFS) από την κορυφή A :

Τελική διάταξη των κορυφών (τα βήματα είναι ίδια με παραπάνω):

A	C	B	E	G	D	H	F
---	---	---	---	---	---	---	---

Αρχικό κόστος, $dist$, από την κορυφή A προς κάθε άλλη και κορυφή που τροποποίησε τελευταία φορά το κόστος προς καθεμία:

A: 0	B: $-\infty$	C: $-\infty$	D: $-\infty$	E: $-\infty$	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: -	C: -	D: -	E: -	F: -	G: -	H: -

Προσπέλαση των κορυφών βάσει της τοπολογικής ταξινόμησης:

- Για την κορυφή A

- Για τη γειτονική της κορυφή B : $dist[B] = dist[A] + w(A, B) = 0 + 0.17 = 0.17$

A: 0	B: 0.17	C: $-\infty$	D: $-\infty$	E: $-\infty$	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: A	C: -	D: -	E: -	F: -	G: -	H: -

- Για τη γειτονική της κορυφή C : $dist[C] = dist[A] + w(A, C) = 0 + 2.40 = 2.40$

A: 0	B: 0.17	C: 2.40	D: $-\infty$	E: $-\infty$	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: -	E: -	F: -	G: -	H: -

- Για τη γειτονική της κορυφή E : $dist[E] = dist[A] + w(A, E) = 0 + 0.27 = 0.27$

A: 0	B: 0.17	C: 2.40	D: $-\infty$	E: 0.27	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: -	E: A	F: -	G: -	H: -

- Για την κορυφή C

- Για τη γειτονική της κορυφή D : $dist[D] = dist[C] + w(C, D) = 2.40 + 1.60 = 4.00$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.27	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: C	E: A	F: -	G: -	H: -

- Για την κορυφή B

- Για τη γειτονική της κορυφή D : καμία αλλαγή
- Για τη γειτονική της κορυφή E : $dist[E] = dist[B] + w(B, E) = 0.17 + 0.64 = 0.81$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: $-\infty$	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: C	E: B	F: -	G: -	H: -

- Για τη γειτονική της κορυφή F : $dist[F] = dist[B] + w(B, F) = 0.17 + 0.20 = 0.37$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 0.37	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: C	E: B	F: B	G: -	H: -

- Για την κορυφή E

- Για τη γειτονική της κορυφή F : $dist[F] = dist[E] + w(E, F) = 0.81 + 1.29 = 2.10$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 2.10	G: $-\infty$	H: $-\infty$
A: -	B: A	C: A	D: C	E: B	F: E	G: -	H: -

– Για τη γειτονική της κορυφή G : $dist[G] = dist[E] + w(E, G) = 0.81 + 1.12 = 1.93$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 2.10	G: 1.93	H: $-\infty$
A: -	B: A	C: A	D: C	E: B	F: E	G: E	H: -

- Για την κορυφή G : καμία γειτονική κορυφή

- Για την κορυφή D

– Για τη γειτονική της κορυφή F : $dist[F] = dist[D] + w(D, F) = 4.00 + 6.64 = 10.64$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 10.64	G: 1.93	H: $-\infty$
A: -	B: A	C: A	D: C	E: B	F: D	G: E	H: -

– Για τη γειτονική της κορυφή H : $dist[H] = dist[D] + w(D, H) = 4.00 + 3.63 = 7.63$

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 10.64	G: 1.93	H: 7.63
A: -	B: A	C: A	D: C	E: B	F: D	G: E	H: D

- Για την κορυφή H : καμία γειτονική κορυφή

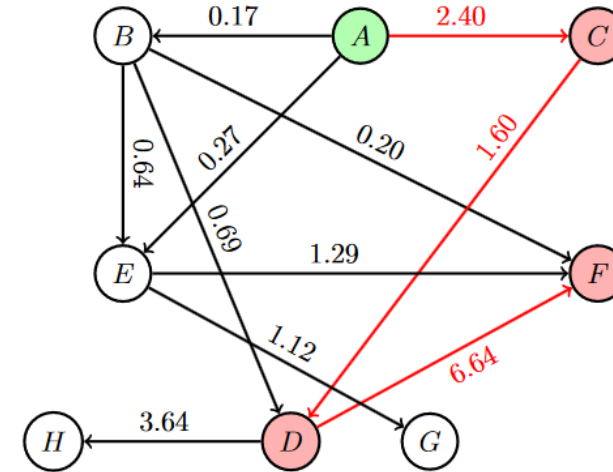
- Για την κορυφή F : καμία γειτονική κορυφή

Τελικός πίνακας:

A: 0	B: 0.17	C: 2.40	D: 4.0	E: 0.81	F: 10.64	G: 1.93	H: 7.63
A: -	B: A	C: A	D: C	E: B	F: D	G: E	H: D

Η μέγιστη τιμή είναι, 10.64, και προκύπτει από το μονοπάτι: $A \rightarrow C \rightarrow D \rightarrow F$:

Σχήμα 6: Μονοπάτι μεγαλύτερου κόστους



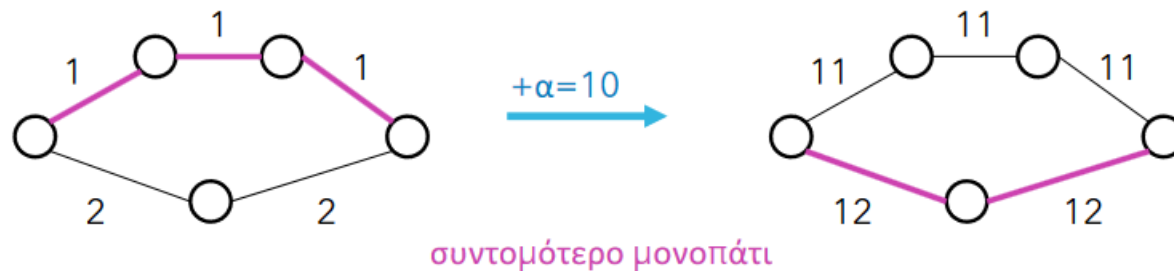
Αποδείξτε αν τα ακόλουθα είναι σωστά ή λαθεμένα;

Σε έναν γράφο με βάρη, το συντομότερο μονοπάτι μεταξύ δύο κορυφών δε μεταβάλλεται αν όλα τα βάρη:

- A. πολλαπλασιαστούν με τον ίδιο θετικό αριθμό
- B. αυξηθούν κατά έναν έναν θετικό αριθμό (πρόσθεση)

A. Είναι σωστό. Για το συντομότερο μονοπάτι αρχικά, ισχύει ότι $\sum_i w_i < \sum_j w_j$, όπου με το πρώτο άθροισμα συμβολίζει το άθροισμα βαρών των ακμών του συντομότερου μονοπατιού και το δεύτερο, το άθροισμα βαρών των ακμών κάθε άλλου μονοπατιού. Όταν λοιπόν οι ακμές πολλαπλασιαστούν με κάποιον θετικό αριθμό, έστω α , τότε θα εξακολουθεί να ισχύει η ανισότητα, καθώς: $\sum_i \alpha \cdot w_i = \alpha(\sum_i w_i) < \sum_j \alpha \cdot w_j = \alpha(\sum_j w_j)$.

B. Είναι λαθεμένο, καθώς υπάρχει το ακόλουθο αντιπαράδειγμα, όπου αποκτά σημασία ο αριθμός των ακμών του αρχικά συντομότερου μονοπατιού.



Πυθαγόρεια 4άδα:

ακέραιοι (a, b, c, d) ,
 $d = (a^2 + b^2 + c^2)^{0.5}$

Αλγόριθμος $O(n^2)$ για αναζήτηση ύπαρξης πυθαγόρειας 4αδας, σε πίνακα n διακριτών θετικών ακεραίων, με δυνατότητα πολλαπλής χρήσης ενός.

- Ισοδυναμεί με την εύρεση 4άδας (a,b,c,d) τέτοιας ώστε:
 $d^2 = a^2 + b^2 + c^2$ ή $(d^2 - c^2) = (a^2 + b^2)$.
- Σχηματίζουμε όλα τα δυνατά διατεταγμένα ζεύγη (x,y) από τους n ακεραίους, που θα έχουν πλήθος n^2 , αφού κάθε ένας από τους n αυτούς αριθμούς θα είναι το πρώτο μέλος σε ζεύγος με καθέναν από τους n αριθμούς (συμπεριλαμβανομένου και του εαυτού του). [$O(n^2)$]
- Για κάθε ζεύγος, υπολογίζεται η τιμή $x^2 + y^2$ (υποψήφια a και b της πιθανής πυθαγόρειας 4άδας), η οποία προστίθεται σε έναν πίνακα κατακερματισμού. [$O(n^2)$]
- Μετά, για κάθε ζεύγος, υπολογίζεται η τιμή $x^2 - y^2$ (υποψήφια d και c της πιθανής πυθαγόρειας 4άδας), η οποία αναζητείται στον πίνακα κατακερματισμού. [$O(n^2)$]
- Αν βρεθεί, τότε γνωρίζουμε ότι υπάρχει πυθαγόρεια 4άδα.
- Διαφορετικά, αν δε βρεθεί για κανένα ζεύγος, δεν υπάρχει.
- Συνολικά λοιπόν, ο αναμενόμενος χρόνος είναι $O(n^2)$.
- Η απάντηση για την έρευση ή μη, δε σημαίνει ότι μας ενδιαφέρει ποια συγκεκριμένα είναι. Ωστόσο, θα μπορούσε σε κάθε θέση του πίνακα κατακερματισμού, να υπάρχει το ζεύγος που οδήγησε σε συγκεκριμένη τιμή του αθροίσματος την πρώτη φορά.

Σχεδιάστε αλγόριθμο, ο οποίος, δοθέντος ενός πίνακα A με n διαφορετικούς ακεραίους στο εύρος $\{1, \dots, n^4\}$, και τιμή στόχο τον ακέραιο x , εκτυπώνει όλα τα ζεύγη αριθμών (i, j) τέτοια ώστε $A[i] + A[j] = x$. Ο αναμενόμενος χρόνος εκτέλεσης πρέπει να είναι $O(n)$.

- Ένας απλός τρόπος, είναι να χρησιμοποιήσουμε, όπως πριν, πίνακα κατακερματισμού για τις τιμές $A[i]$ και $(x - A[i])$.
- Πρώτα, τοποθετούμε τις τιμές $A[i]$ στους αντίστοιχους κάδους.
- Στη συνέχεια, βρίσκουμε τον κάδο που αντιστοιχεί στην τιμή $b = (x - A[i])$, και για κάθε «σύγκρουση», δηλαδή εύρεση στον ίδιο κάδο, ελέγχουμε αν για την τιμή a που υπήρχε εκεί, $a + b = x$.

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ5,6:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος
προηγούμενων ετών, Michael Sambol, SMA5503